eShop Project Summary

Oct 11, 2026 · @Perman Mammedov

Overview

eShop is a full-stack online shop that you built over 10-11 October 2026 to prove you can build and explain a whole application on your own. It has a Django REST API backend, a React frontend, and automated tests. The code is public at github.com/pmammedov/eshop.

Working today: a Product model and admin page, a product list and product detail API, a React page that shows the products, and API tests. Planned: cart, Stripe test checkout, TypeScript, PostgreSQL, Docker and a live deployment.

Tech stack

The project runs on Python and JavaScript, with SQLite as the development database.

Layer	Tool	Role in the project
Backend language	Python 3.12	Runs the Django code
Backend framework	Django 6.1	Models, admin, URL routing
API	Django REST Framework	Serializers and API views
Cross-origin access	django-cors-headers	Lets React (port 5173) call Django (port 8000)
Database	SQLite (db.sqlite3)	Stores products during development
Frontend	React with Vite 8 (JavaScript)	Product list page
Runtime	Node 24 and npm	Runs the Vite dev server
Version control	Git 2.43 and GitHub CLI	Commits, pushes and GitHub login
Editor	VS Code	Writing the code

Environment setup

You set up the whole project from an empty folder at `~/Documents/projects/eshop`, with the backend and frontend in one repository.

1. Checked that Python 3, Node and Git were installed (on a Mac the command is `python3`, not `python`).
2. Created the repo with `git init`, switched the default branch to `main`, and set your Git name and email.
3. Created a Python virtual environment in `backend/.venv` and installed Django, Django REST Framework and django-cors-headers.
4. Started the Django project (`config`) and the `products` app.
5. Created the React app in `frontend` with Vite.
6. Added a `.gitignore` for `.venv`, `node_modules`, `__pycache__`, `db.sqlite3`, `.DS_Store` and `.env`.
7. Logged in to GitHub with `gh auth login`, because GitHub no longer accepts account passwords for `git push`.
8. Created the public `eshop` repository on GitHub from the terminal and pushed the first commit.

Backend (Django)

The backend stores products and serves them as JSON through two read-only endpoints.

Product model (`products/models.py`)

Field	Type	Note
name	CharField	Max 200 characters
description	TextField	Long text
price	DecimalField	8 digits, 2 decimal places; exact, unlike a float
stock	PositiveIntegerField	Whole number, 0 or more
image_url	URLField	Optional
created_at	DateTimeField	Filled in automatically

What you built around it

- Applied migrations with `makemigrations` and `migrate`, and created a superuser.

- Registered `Product` in the Django admin and added four products there (headphones, keyboard, USB-C hub, laptop stand) with placeholder images.
- Wrote `ProductSerializer`, which turns a product into JSON.
- Wrote `ProductListView` and `ProductDetailView` using Django REST Framework's ready-made list and retrieve views.
- Configured CORS so the React app on port 5173 may call the API, and checked it with a `curl` request that returned the `access-control-allow-origin` header.

Endpoint	Returns
`/api/products/`	All products
`/api/products/<id>/`	One product, or a 404 if it does not exist

Frontend (React)

The React page in `frontend/src/App.jsx` fetches the product list from the Django API and shows each product as a card in a responsive grid.

- `useState` holds three values: the products, a loading flag and any error message.
- `useEffect` with an empty dependency list runs the `fetch` once, when the page first loads.
- `fetch` calls `http://127.0.0.1:8000/api/products/`, and `.json()` turns the response into a JavaScript array.
- `products.map(...)` renders one card per product, and `key={product.id}` lets React track each card.
- The page shows "Loading products..." while waiting and an error message if the request fails.
- `App.css` uses CSS grid (`repeat(auto-fill, minmax(240px, 1fr))`) for the layout. You emptied `index.css` so the Vite starter styles would not interfere.

When the Django server is not running, the page shows an error instead of products. That behaviour is the error handling working as designed.

Testing

Four API tests pass (`python manage.py test` reported `Ran 4 tests ... OK`). They live in `products/tests.py` and use Django REST Framework's `APITestCase`, which runs against a temporary database, so your real products are never touched.

Test	Checks
test_list_returns_200	The list endpoint responds successfully
test_list_returns_products	One product with the right name and price comes back
test_detail_returns_product	The detail endpoint returns the right product
test_detail_returns_404_for_missing_product	An unknown ID returns 404

You built the detail endpoint **test-first**. The new detail test failed with `404 != 200` (red), then passed once you added `ProductDetailView` and its URL (green). Two problems along the way taught you useful habits: an `IndentationError` from a mis-indented test, and a test result that did not change because `urls.py` had not been saved. Reading the error and checking the file on disk (`cat`) is how you found it.

Git history and commands

You confirmed these three commits pushed to GitHub. The React page, the tests and the detail endpoint were committed afterwards; run `git log --oneline` and `git status` to confirm they are on GitHub too.

1. Initial Django and React setup
2. Add Product model, admin and products list API
3. Configure CORS for the React dev server

Command	What it does
`source .venv/bin/activate`	Turns on the Python virtual environment (needed in every new terminal tab)
`python manage.py runserver`	Starts the Django server on port 8000
`python manage.py makemigrations` and `migrate`	Create and apply database changes
`python manage.py createsuperuser`	Creates the admin login
`python manage.py test`	Runs the automated tests
`npm run dev`	Starts the React server on port 5173

Command	What it does
`git add .`, `git commit -m "..."`, `git push`	Stage, save and upload changes
`gh auth login`	Signs in to GitHub from the terminal

Your working setup uses three terminal tabs: Django server, React server, and one for Git and tests.

Interview talking points

Interviewers will ask you to walk through any file in the repo, so practise explaining each of these in your own words.

- **Why DecimalField for price:** floats cannot store some decimals exactly (0.1 + 0.2 gives 0.30000000000000004), and money must be exact.
- **What a serializer does:** converts a model object into JSON for the API, and `fields` controls which columns are exposed.
- **What CORS is:** a browser blocks a page on one origin (port 5173) from reading data from another (port 8000) unless the server allows it. You configured the allowed origin and verified the header with `curl`.
- **How React loads data:** `useEffect` with `[]` fetches once on page load, `useState` stores the result, and `map` with a `key` renders the list.
- **Why a virtual environment:** it keeps the project's Python packages separate from the rest of the machine.
- **What migrations are:** versioned descriptions of database changes that Django applies with `migrate`.
- **Test-driven development:** you wrote the failing detail test first, watched it fail with a 404, then wrote the code that made it pass.
- **Debugging stories:** the missing-save `urls.py` and the `IndentationError` are real examples of how you diagnose a problem from its output.

Your course projects on GitHub were group work, so this repo is where you show what you can build yourself. Be ready to explain every file and every line you type, and say honestly which parts you learned during this project.

Next steps

- [ ] Make sure every commit is pushed (`git status`, `git log --oneline`)
- [ ] Build a React product detail page using `/api/products/<id>/`

- [ ] Add cart logic
- [ ] Add Stripe checkout in test mode
- [ ] Add a README with screenshots, setup steps and a "what I built" section
- [ ] Add a description to the GitHub repo and pin it on your profile
- [ ] Move the database to PostgreSQL
- [ ] Convert the React code to TypeScript
- [ ] Add Docker and deploy a live version
- [ ] Add product categories (books, electronics) and a filter